In [2]:
```
# Step 1: Importing Libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

In [3]:
```
# Step 2: Load the raw dataset

df = pd.read_csv("../data/raw/signal_metrics.csv")

print("Dataset loaded successfully!")
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
```

```
Dataset loaded successfully!
Number of rows: 16829
Number of columns: 12
```

In [4]:
```
# Step 3: Display the first few rows of the dataset
df.head()
```

Out[4]: Columns 1–8 of 13

	Timestamp	Locality	Latitude	Longitude	Signal Strength (dBm)	Signal Quality (%)	Data Throughput (Mbps)
0	50:40.0	Anisabad	25.599109	85.137355	-84.274113	0	1.863890
1	53:47.2	Fraser Road	25.433286	85.070053	-97.653121	0	5.132296
2	56:54.4	Boring Canal Road	25.498809	85.211371	-87.046134	0	1.176985
3	00:01.6	Danapur	25.735138	85.208400	-94.143159	0	68.596932
4	03:08.8	Phulwari Sharif	25.538556	85.159860	-94.564765	0	38.292038

Columns 9–13 of 13 · first column repeated

	Latency (ms)	Network Type	BB60C Measurement (dBm)	srsRAN Measurement (dBm)	BladeRFxA9 Measurement (dBm)
0	129.122914	3G	0.000000	0.000000	0.000000
1	54.883606	4G	-95.810791	-105.452359	-99.920892
2	119.598286	LTE	-91.593861	-95.419482	-87.714070
3	46.598387	5G	-90.642773	-101.895905	-96.570698
4	30.342828	5G	-90.489100	-103.318303	-95.102467

In [5]:
```
# Step 4: Display the column names of the dataset
print(df.columns.tolist())
```

```
['Timestamp', 'Locality', 'Latitude', 'Longitude', 'Signal Strength (dBm)',
'Signal Quality (%)', 'Data Throughput (Mbps)', 'Latency (ms)', 'Network Type',
'BB60C Measurement (dBm)', 'srsRAN Measurement (dBm)', 'BladeRFxA9 Measurement
(dBm)']
```

In [6]:
```
# Step 5: Display the data types and non-null counts of the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16829 entries, 0 to 16828
Data columns (total 12 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   Timestamp                     16829 non-null  object
 1   Locality                      16829 non-null  object
 2   Latitude                      16829 non-null  float64
 3   Longitude                     16829 non-null  float64
 4   Signal Strength (dBm)         16829 non-null  float64
 5   Signal Quality (%)            16829 non-null  int64
 6   Data Throughput (Mbps)        16829 non-null  float64
 7   Latency (ms)                  16829 non-null  float64
 8   Network Type                  16829 non-null  object
 9   BB60C Measurement (dBm)       16829 non-null  float64
 10  srsRAN Measurement (dBm)      16829 non-null  float64
 11  BladeRFxA9 Measurement (dBm)  16829 non-null  float64
dtypes: float64(8), int64(1), object(3)
memory usage: 1.5+ MB
```

In [7]:
```
# Step 6: Check for missing values in the dataset
print(df.isnull().sum())
```

```
Timestamp                     0
Locality                      0
Latitude                      0
Longitude                     0
Signal Strength (dBm)         0
Signal Quality (%)            0
Data Throughput (Mbps)        0
Latency (ms)                  0
Network Type                  0
BB60C Measurement (dBm)       0
srsRAN Measurement (dBm)      0
BladeRFxA9 Measurement (dBm)  0
dtype: int64
```

In [8]:
```
# Step 7: Check for duplicate rows in the dataset
print("Duplicate rows:", df.duplicated().sum())
```

```
Duplicate rows: 0
```

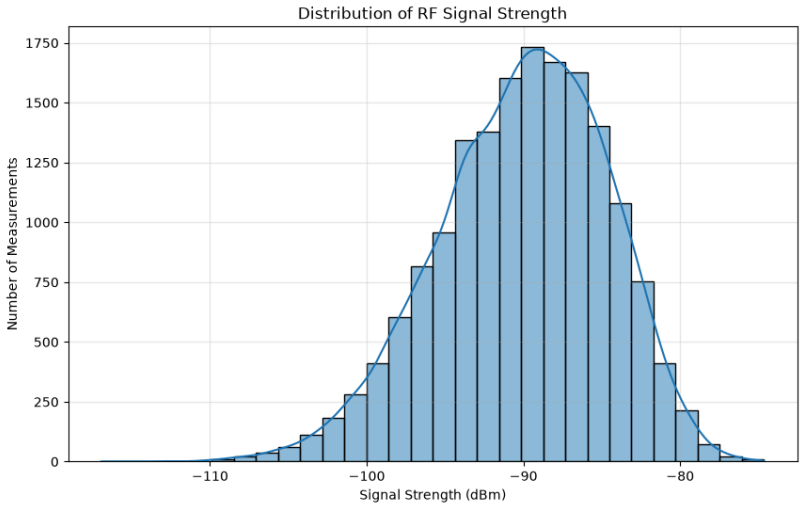
In [9]:
```
# Step 8: Signal Strength Distribution

plt.figure(figsize=(10, 6))

sns.histplot(
    df["Signal Strength (dBm)"],
    bins=30,
    kde=True
)

plt.title("Distribution of RF Signal Strength")
plt.xlabel("Signal Strength (dBm)")
plt.ylabel("Number of Measurements")
plt.grid(alpha=0.3)

plt.show()
```



In [10]:

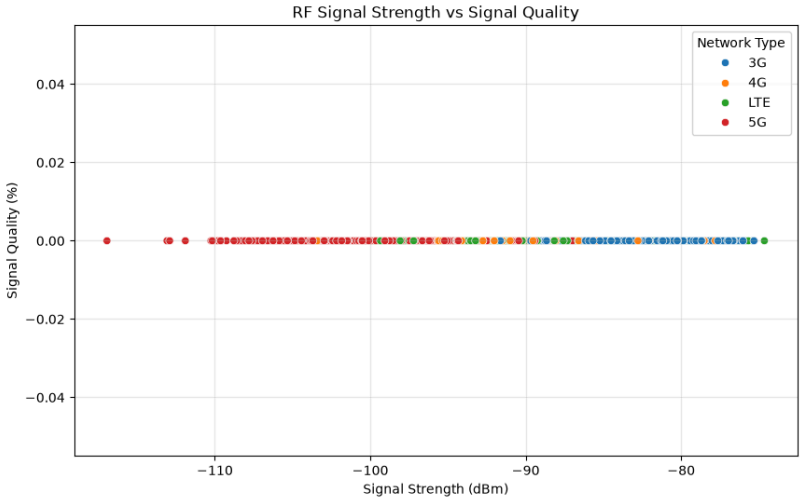
```
# Step 9: Signal Strength vs Signal Quality

plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="Signal Strength (dBm)",
    y="Signal Quality (%)",
    hue="Network Type"
)

plt.title("RF Signal Strength vs Signal Quality")
plt.xlabel("Signal Strength (dBm)")
plt.ylabel("Signal Quality (%)")
plt.grid(alpha=0.3)

plt.show()
```



In [11]:

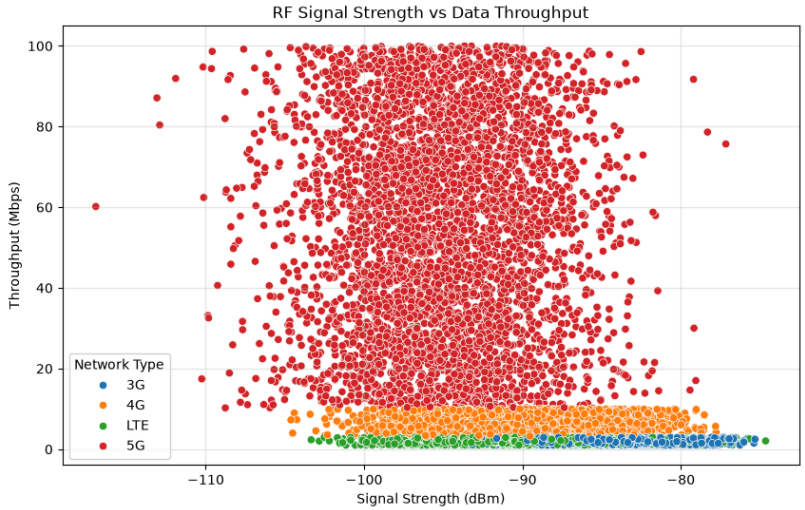
```
# Step 10: Signal Strength vs Throughput

plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="Signal Strength (dBm)",
    y="Data Throughput (Mbps)",
    hue="Network Type"
)

plt.title("RF Signal Strength vs Data Throughput")
plt.xlabel("Signal Strength (dBm)")
plt.ylabel("Throughput (Mbps)")
plt.grid(alpha=0.3)

plt.show()
```



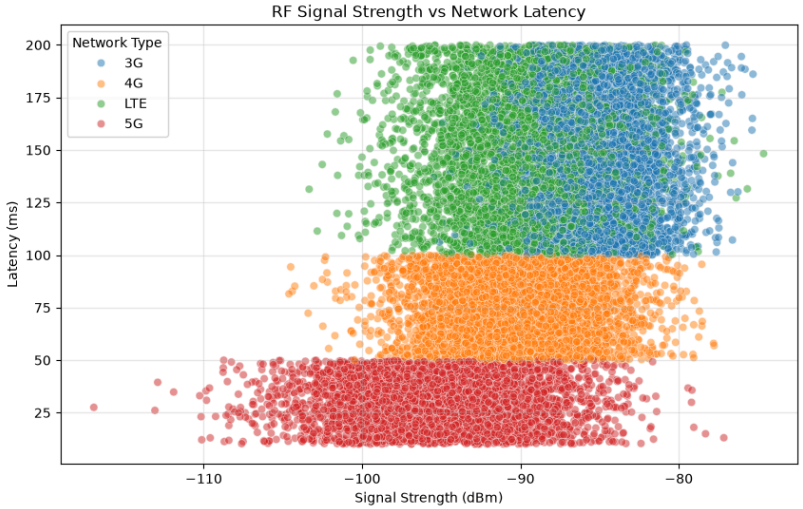
In [12]:
```
# Step 11: RF Signal Strength vs Latency

plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="Signal Strength (dBm)",
    y="Latency (ms)",
    hue="Network Type",
    alpha=0.5
)

plt.title("RF Signal Strength vs Network Latency")
plt.xlabel("Signal Strength (dBm)")
plt.ylabel("Latency (ms)")
plt.grid(alpha=0.3)

plt.show()
```



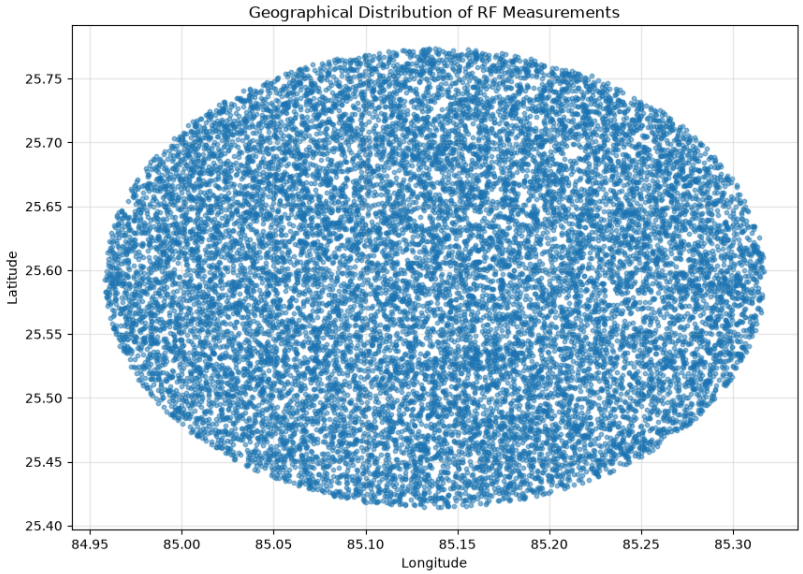
In [13]:
```
# Step 12: Geographical distribution of RF measurements

plt.figure(figsize=(10, 7))

plt.scatter(
    df["Longitude"],
    df["Latitude"],
    s=10,
    alpha=0.5
)

plt.title("Geographical Distribution of RF Measurements")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.grid(alpha=0.3)

plt.show()
```



In [14]:

```
# Step 13: Geographical RF Signal Strength Map

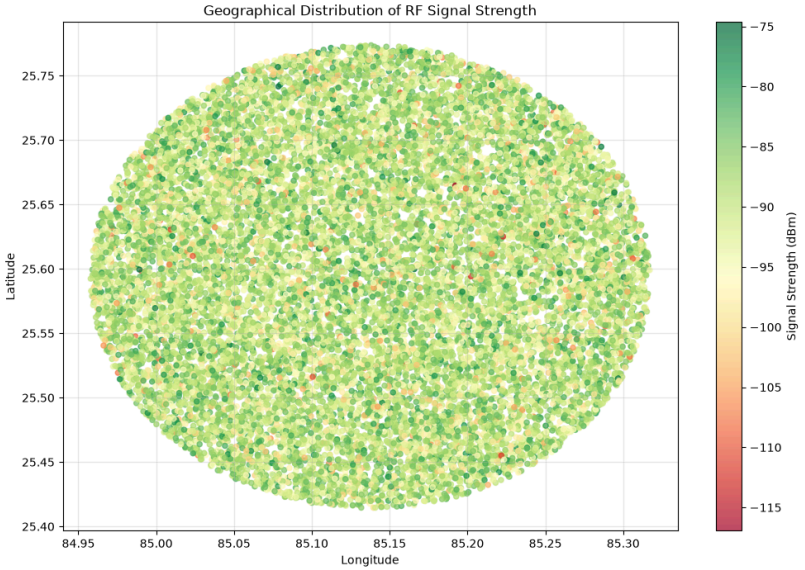
plt.figure(figsize=(12, 8))

scatter = plt.scatter(
    df["Longitude"],
    df["Latitude"],
    c=df["Signal Strength (dBm)"],
    cmap="RdYlGn",
    s=20,
    alpha=0.7
)

plt.colorbar(scatter, label="Signal Strength (dBm)")

plt.title("Geographical Distribution of RF Signal Strength")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.grid(alpha=0.3)

plt.show()
```



In [15]:

```
# Step 14: Signal strength statistics

print("Minimum Signal Strength:",
      df["Signal Strength (dBm)"].min())

print("Maximum Signal Strength:",
      df["Signal Strength (dBm)"].max())

print("Mean Signal Strength:",
      df["Signal Strength (dBm)"].mean())

print("Median Signal Strength:",
      df["Signal Strength (dBm)"].median())

print("\nPercentiles:")
print(df["Signal Strength (dBm)"].quantile([0.25, 0.50, 0.75]))
```

```
Minimum Signal Strength: -116.9422667
Maximum Signal Strength: -74.64484797
Mean Signal Strength: -90.07248449578763
Median Signal Strength: -89.66556554

Percentiles:
0.25   -93.615962
0.50   -89.665566
0.75   -86.145491
Name: Signal Strength (dBm), dtype: float64
```

In [16]:

```python
# Step 15.1: Classify RF signal strength

q25 = df["Signal Strength (dBm)"].quantile(0.25)
q75 = df["Signal Strength (dBm)"].quantile(0.75)

def classify_signal(signal):
    if signal ⩽ q25:
        return "Weak"
    elif signal ⩽ q75:
        return "Moderate"
    else:
        return "Strong"

df["Signal Category"] = df["Signal Strength (dBm)"].apply(classify_signal)

print("25th Percentile:", q25)
print("75th Percentile:", q75)

print("\nSignal Category Counts:")
print(df["Signal Category"].value_counts())
```

```
25th Percentile: -93.61596247
75th Percentile: -86.14549103

Signal Category Counts:
Signal Category
Moderate    8414
Weak        4208
Strong      4207
Name: count, dtype: int64
```

In [17]:

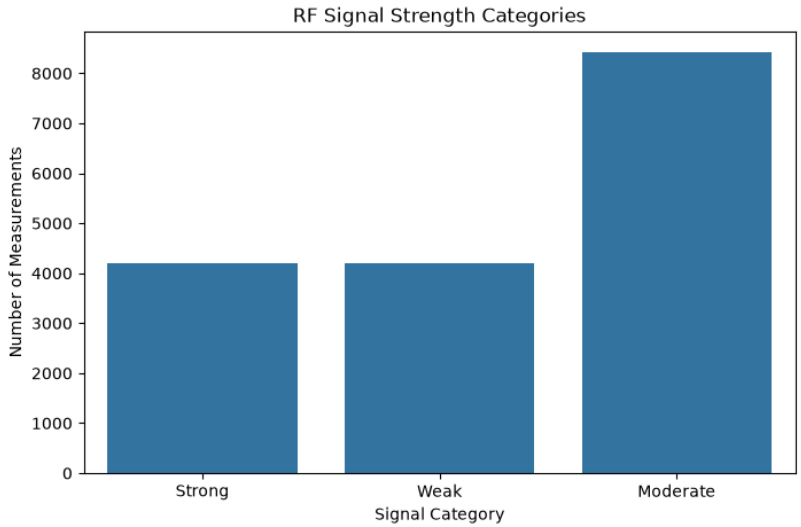
```python
# Step 15.2: Visualize signal categories

plt.figure(figsize=(8, 5))

sns.countplot(
    data=df,
    x="Signal Category"
)

plt.title("RF Signal Strength Categories")
plt.xlabel("Signal Category")
plt.ylabel("Number of Measurements")

plt.show()
```



In [18]:

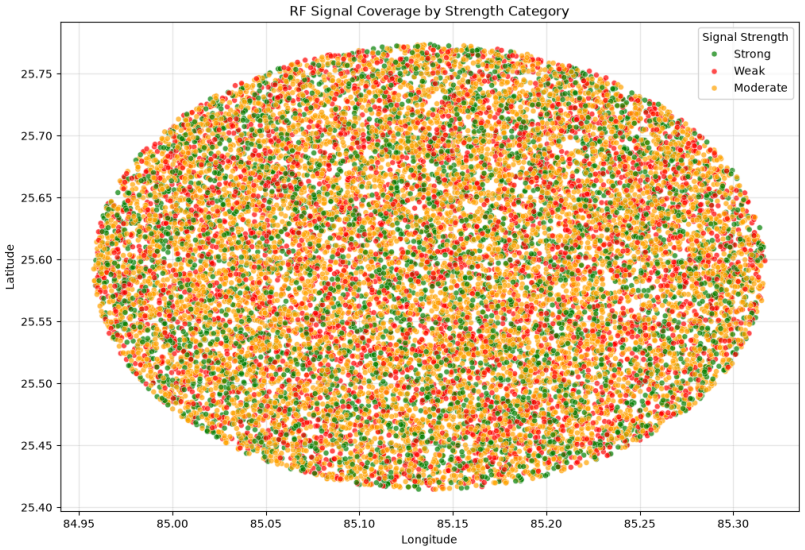
```
# Step 16: Geographical RF Coverage by Signal Category

plt.figure(figsize=(12, 8))

sns.scatterplot(
    data=df,
    x="Longitude",
    y="Latitude",
    hue="Signal Category",
    palette={
        "Weak": "red",
        "Moderate": "orange",
        "Strong": "green"
    },
    s=25,
    alpha=0.7
)

plt.title("RF Signal Coverage by Strength Category")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.legend(title="Signal Strength")
plt.grid(alpha=0.3)

plt.show()
```



In [19]:

```
# Step 17: Create RF Signal Heat Map

from scipy.interpolate import griddata
import numpy as np
import matplotlib.pyplot as plt

# Extract latitude, longitude and signal strength directly from df
lat = df["Latitude"].values
lon = df["Longitude"].values
signal = df["Signal Strength (dBm)"].values

# Create a geographical grid
grid_lon = np.linspace(lon.min(), lon.max(), 200)
grid_lat = np.linspace(lat.min(), lat.max(), 200)

grid_lon, grid_lat = np.meshgrid(grid_lon, grid_lat)

# Interpolate signal strength over the geographical grid
grid_signal = griddata(
    (lon, lat),
    signal,
    (grid_lon, grid_lat),
    method="linear"
)

# Create the heat map
plt.figure(figsize=(12, 8))

heatmap = plt.contourf(
    grid_lon,
    grid_lat,
    grid_signal,
    levels=30,
    cmap="RdYlGn"
)

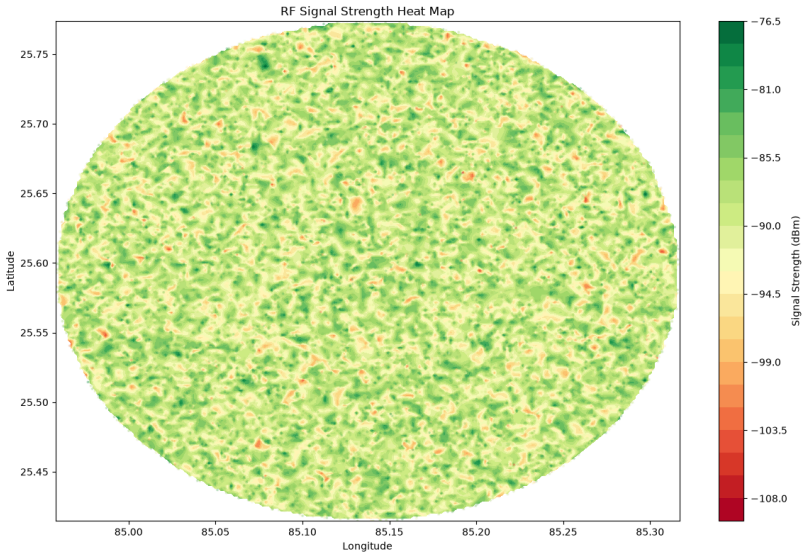
# Add the original measurement locations
plt.scatter(
    lon,
    lat,
    c=signal,
    cmap="RdYlGn",
    s=3,
    alpha=0.25
)

# Color bar
plt.colorbar(
```

```
    heatmap,
    label="Signal Strength (dBm)"
)

plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("RF Signal Strength Heat Map")

plt.tight_layout()
plt.show()
```



In [20]:

```
# Step 18: Compare Actual vs Interpolated Signal Strength

# Interpolate the heat-map value at every original measurement location
predicted_signal = griddata(
    (lon, lat),
    signal,
    (lon, lat),
    method="linear"
)

# Remove locations where interpolation produced NaN
valid = ~np.isnan(predicted_signal)

actual = signal[valid]
predicted = predicted_signal[valid]

# Calculate error
mae = np.mean(np.abs(actual - predicted))
rmse = np.sqrt(np.mean((actual - predicted) ** 2))

print("Heat Map Validation")
print("-------------------")
print("Valid measurements:", len(actual))
print("Mean Absolute Error (MAE):", round(mae, 3), "dBm")
print("Root Mean Square Error (RMSE):", round(rmse, 3), "dBm")
```

```
Heat Map Validation
-------------------
Valid measurements: 16829
Mean Absolute Error (MAE): 0.0 dBm
Root Mean Square Error (RMSE): 0.0 dBm
```

In [21]:

```
# Step 19: Identify Strongest and Weakest RF Locations

# Strongest RF signals (closest to 0 dBm)
strongest = df.nlargest(10, "Signal Strength (dBm)")[
    ["Locality", "Latitude", "Longitude", "Signal Strength (dBm)"]
]

# Weakest RF signals
weakest = df.nsmallest(10, "Signal Strength (dBm)")[
    ["Locality", "Latitude", "Longitude", "Signal Strength (dBm)"]
]

print("TOP 10 STRONGEST RF LOCATIONS")
print("-------------------------------")
print(strongest.to_string(index=False))

print("\nTOP 10 WEAKEST RF LOCATIONS")
print("----------------------------")
print(weakest.to_string(index=False))
```

```
TOP 10 STRONGEST RF LOCATIONS
-------------------------------
       Locality  Latitude  Longitude  Signal Strength (dBm)
        Kumhrar 25.534256  85.028930             -74.644848
  Gandhi Maidan 25.599504  84.989190             -75.314531
    Pataliputra 25.599897  85.165807             -75.349049
Exhibition Road 25.741651  85.078505             -75.381195
 Rajendra Nagar 25.566981  85.121491             -75.672853
      Bankipore 25.649741  85.297338             -75.814159
    Fraser Road 25.696077  85.114397             -76.018573
Phulwari Sharif 25.628203  85.096641             -76.040852
        Danapur 25.625088  85.260116             -76.047085
    Pataliputra 25.515071  85.296669             -76.266148

TOP 10 WEAKEST RF LOCATIONS
----------------------------
         Locality  Latitude  Longitude  Signal Strength (dBm)
Patliputra Colony 25.664713  85.192150            -116.942267
       Kankarbagh 25.625183  85.222844            -113.082820
  Exhibition Road 25.594323  85.202219            -112.907955
      Bailey Road 25.455075  85.221732            -111.899047
       Kidwaipuri 25.630599  85.009329            -110.253857
          Danapur 25.516195  85.100526            -110.167665
       Kidwaipuri 25.525903  85.160490            -110.127088
      Boring Road 25.736879  85.156717            -109.858119
        Anandpuri 25.756864  85.192869            -109.824672
        S.K. Puri 25.610025  85.023490            -109.636501
```

In [22]:

```
# Step 20.1: RF Signal Strength by Network Type

network_summary = df.groupby("Network Type")["Signal Strength (dBm)"].agg(
    ["count", "mean", "min", "max", "std"]
)

print("RF Signal Strength by Network Type")
print("-----------------------------------")
print(network_summary.round(2))
```

```
RF Signal Strength by Network Type
-----------------------------------
              count    mean     min    max   std
Network Type
3G             4208 -85.01  -95.42 -75.31  3.03
4G             4219 -90.17 -104.62 -77.77  4.01
5G             4178 -95.15 -116.94 -77.16  4.95
LTE            4224 -90.00 -103.34 -74.64  3.97
```

In [23]:

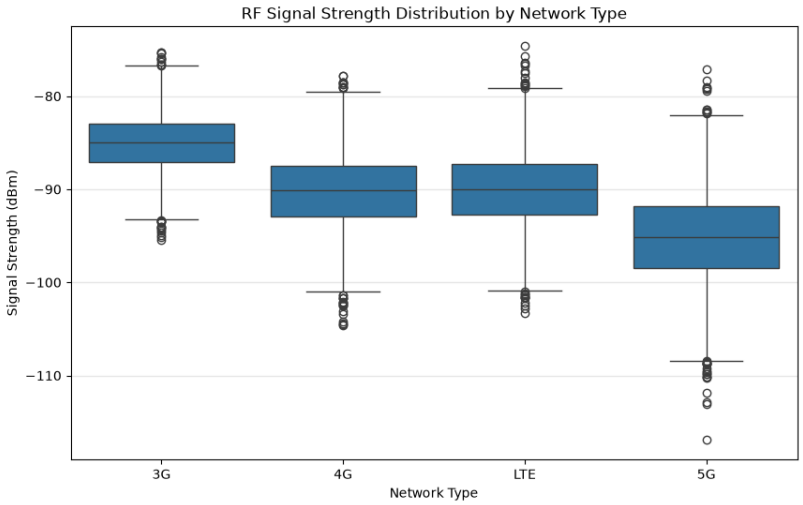
```
# Step 20.2: Box plot of RF signal strength for each network type

plt.figure(figsize=(10, 6))

sns.boxplot(
    data=df,
    x="Network Type",
    y="Signal Strength (dBm)"
)

plt.title("RF Signal Strength Distribution by Network Type")
plt.xlabel("Network Type")
plt.ylabel("Signal Strength (dBm)")
plt.grid(axis="y", alpha=0.3)

plt.show()
```



In [24]:

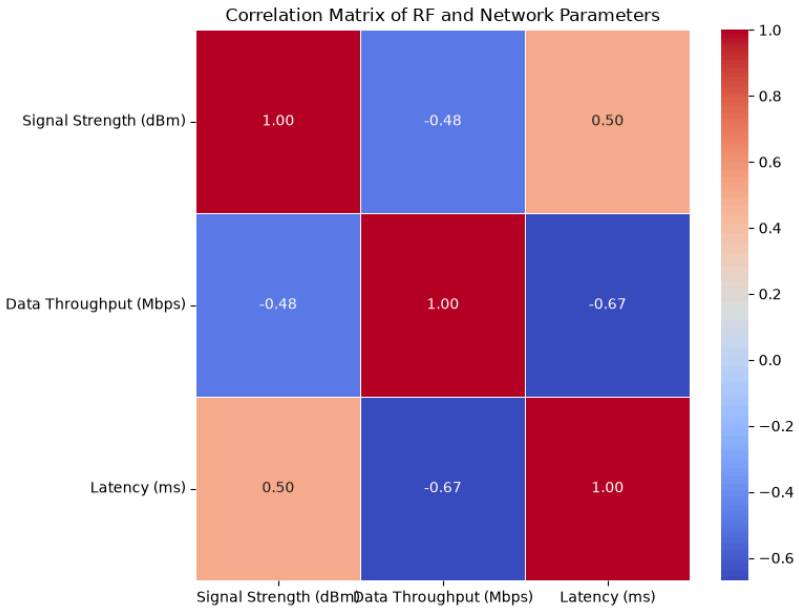
```
# Step 21: Correlation Heatmap

plt.figure(figsize=(8, 6))

correlation = df[
    [
        "Signal Strength (dBm)",
        "Data Throughput (Mbps)",
        "Latency (ms)"
    ]
].corr()

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)

plt.title("Correlation Matrix of RF and Network Parameters")
plt.tight_layout()
plt.show()
```



In [25]:

```python
# Step 22: Identify Strongest and Weakest RF Regions

# Create geographical grid cells
df["Lat_Grid"] = df["Latitude"].round(2)
df["Lon_Grid"] = df["Longitude"].round(2)

# Calculate average signal strength in each geographical region
region_signal = (
    df.groupby(["Lat_Grid", "Lon_Grid"])["Signal Strength (dBm)"]
    .agg(["mean", "count"])
    .reset_index()
)

# Rename columns
region_signal.columns = [
    "Latitude",
    "Longitude",
    "Average Signal (dBm)",
    "Measurements"
]

# Sort from strongest to weakest
strongest_regions = region_signal.sort_values(
    "Average Signal (dBm)",
    ascending=False
).head(10)

weakest_regions = region_signal.sort_values(
    "Average Signal (dBm)",
    ascending=True
).head(10)

print("TOP 10 STRONGEST RF REGIONS")
print("----------------------------")
print(strongest_regions.to_string(index=False))

print("\nTOP 10 WEAKEST RF REGIONS")
print("--------------------------")
print(weakest_regions.to_string(index=False))
```

```
TOP 10 STRONGEST RF REGIONS
----------------------------
 Latitude  Longitude  Average Signal (dBm)  Measurements
    25.50      84.98            -83.717538             3
    25.74      85.08            -84.348069             9
    25.64      84.96            -84.688474             3
    25.41      85.13            -85.432328             1
    25.44      85.04            -85.648557             2
```

```
    25.57      85.12            -85.700839            13
    25.54      85.02            -85.746013            15
    25.67      85.22            -85.952821            11
    25.69      85.12            -86.189574            16
    25.42      85.19            -86.206901             6

TOP 10 WEAKEST RF REGIONS
--------------------------
 Latitude  Longitude  Average Signal (dBm)  Measurements
    25.75      85.04           -103.580589             1
    25.76      85.21            -98.640485             3
    25.69      84.98            -98.004196             1
    25.75      85.23            -96.728359             3
    25.77      85.08            -95.855523             1
    25.41      85.14            -95.390245             2
    25.77      85.09            -95.106078             2
    25.52      85.10            -95.051234            10
    25.60      85.29            -94.835770            17
    25.54      85.08            -94.710342            15
```

In [26]:

```python
# Step 23: Regional Average RF Signal Map

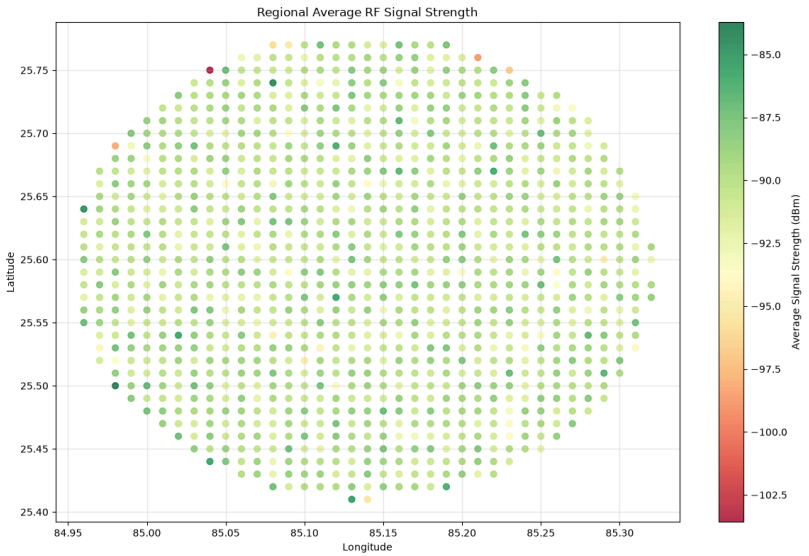
plt.figure(figsize=(12, 8))

scatter = plt.scatter(
    region_signal["Longitude"],
    region_signal["Latitude"],
    c=region_signal["Average Signal (dBm)"],
    cmap="RdYlGn",
    s=40,
    alpha=0.8
)

plt.colorbar(
    scatter,
    label="Average Signal Strength (dBm)"
)

plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("Regional Average RF Signal Strength")

plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



In [27]:

```
# Step 24: Interactive RF Coverage Map

import folium
from folium.plugins import MarkerCluster
import os

# ---------------------------------------------------
# 1. Calculate center of the dataset
# ---------------------------------------------------

center_lat = df["Latitude"].mean()
center_lon = df["Longitude"].mean()

# ---------------------------------------------------
# 2. Create map WITHOUT default tiles
# ---------------------------------------------------

rf_map = folium.Map(
    location=[center_lat, center_lon],
    zoom_start=11,
    tiles=None
)

# ---------------------------------------------------
# 3. Add Esri World Street Map
# ---------------------------------------------------

folium.TileLayer(
    tiles="https://server.arcgisonline.com/ArcGIS/rest/services/"
          "World_Street_Map/MapServer/tile/{z}/{y}/{x}",
    attr="Esri",
    name="Esri World Street Map",
    overlay=False,
    control=True
).add_to(rf_map)

# ---------------------------------------------------
# 4. Create marker cluster
# ---------------------------------------------------

marker_cluster = MarkerCluster(
    name="RF Measurements"
).add_to(rf_map)

# ---------------------------------------------------
# 5. Function to assign signal colors
# ---------------------------------------------------
```

```
def signal_color(category):

    if category == "Strong":
        return "green"

    elif category == "Moderate":
        return "orange"

    else:
        return "red"

# ---------------------------------------------------
# 6. Add RF measurements
# ---------------------------------------------------

for _, row in df.iterrows():

    color = signal_color(row["Signal Category"])

    folium.CircleMarker(
        location=[
            row["Latitude"],
            row["Longitude"]
        ],

        radius=3,

        color=color,

        fill=True,

        fill_color=color,

        fill_opacity=0.7,

        popup=(
            f"<b>RF Signal Measurement</b><br>"
            f"Signal Strength: "
            f"{row['Signal Strength (dBm)']:.2f} dBm<br>"
            f"Signal Category: "
            f"{row['Signal Category']}<br>"
            f"Network Type: "
            f"{row['Network Type']}<br>"
            f"Latitude: "
            f"{row['Latitude']:.6f}<br>"
            f"Longitude: "
            f"{row['Longitude']:.6f}"
        )
```

```
    ).add_to(marker_cluster)

# ----------------------------------------------------
# 7. Add layer control
# ----------------------------------------------------

folium.LayerControl().add_to(rf_map)

# ----------------------------------------------------
# 8. Save using a NEW filename
# ----------------------------------------------------

output_path = "outputs/maps/rf_coverage_map_esri.html"

os.makedirs("outputs/maps", exist_ok=True)

rf_map.save(output_path)

print("RF Coverage Map created successfully!")
print("Saved to:", output_path)
print("File size:",
      round(os.path.getsize(output_path) / 1024 / 1024, 2),
      "MB")

# ----------------------------------------------------
# 9. Display map
# ----------------------------------------------------

rf_map
```

```
RF Coverage Map created successfully!
Saved to: outputs/maps/rf_coverage_map_esri.html
File size: 18.98 MB
```

Out[27]: Make this Notebook Trusted to load map: File → Trust Notebook

In [28]:
```
# Step 25: Check Normality of RF Signal Strength

from scipy.stats import normaltest
import matplotlib.pyplot as plt
import seaborn as sns

# Extract signal strength values
signal = df["Signal Strength (dBm)"].dropna()

# Visualize the distribution
plt.figure(figsize=(10, 6))

sns.histplot(
    signal,
    bins=30,
    kde=True
)

plt.title("Distribution of RF Signal Strength")
plt.xlabel("Signal Strength (dBm)")
plt.ylabel("Frequency")
plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()

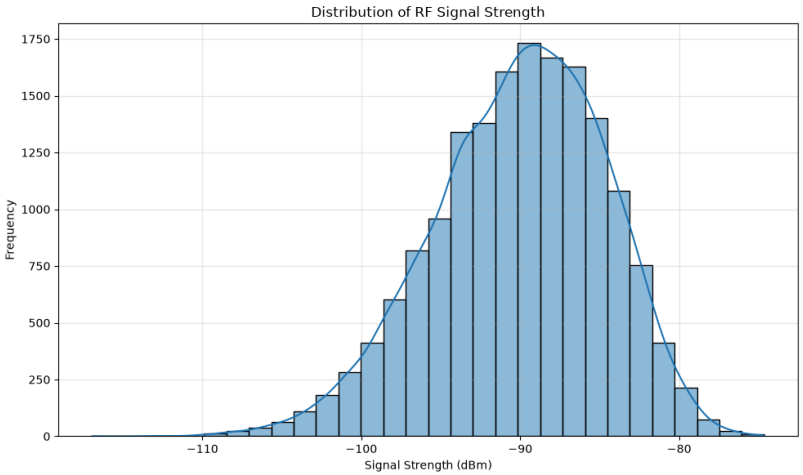
# Calculate statistical measures
print("Mean:", round(signal.mean(), 3))
print("Median:", round(signal.median(), 3))
print("Standard Deviation:", round(signal.std(), 3))

# Perform D'Agostino-Pearson normality test
statistic, p_value = normaltest(signal)

print("\nNormality Test")
print("--------------------------")
print("Test Statistic:", round(statistic, 3))
print("P-value:", p_value)

# Interpret the result
alpha = 0.05

if p_value < alpha:
    print("Conclusion: Evidence against a normal distribution.")
else:
    print("Conclusion: Insufficient evidence to reject normality.")
```




```
Mean: -90.072
Median: -89.666
Standard Deviation: 5.399

Normality Test
----------------------------
Test Statistic: 384.534
P-value: 3.15883879287517e-84
Conclusion: Evidence against a normal distribution.
```

In [29]:

```python
# Step 26: Detect Outliers Using the IQR Method

signal = df["Signal Strength (dBm)"].dropna()

# Calculate quartiles
Q1 = signal.quantile(0.25)
Q3 = signal.quantile(0.75)

# Calculate interquartile range
IQR = Q3 - Q1

# Define outlier boundaries
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify potential outliers
outliers = signal[
    (signal < lower_bound) |
    (signal > upper_bound)
]

print("Q1:", round(Q1, 3), "dBm")
print("Q3:", round(Q3, 3), "dBm")
print("IQR:", round(IQR, 3), "dB")

print("\nLower Bound:", round(lower_bound, 3), "dBm")
print("Upper Bound:", round(upper_bound, 3), "dBm")

print("\nTotal Measurements:", len(signal))
print("Potential Outliers:", len(outliers))
print(
    "Outlier Percentage:",
    round(len(outliers) / len(signal) * 100, 2),
    "%"
)

# Visualize potential outliers
plt.figure(figsize=(10, 4))

sns.boxplot(x=signal)

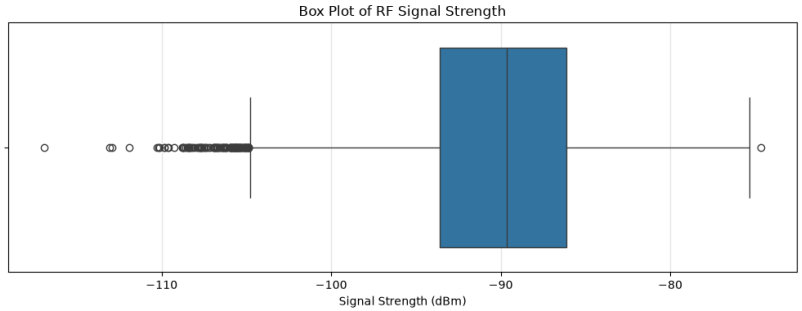
plt.title("Box Plot of RF Signal Strength")
plt.xlabel("Signal Strength (dBm)")
plt.grid(axis="x", alpha=0.3)

plt.tight_layout()
plt.show()
```

```
Q1: -93.616 dBm
Q3: -86.145 dBm
IQR: 7.47 dB

Lower Bound: -104.822 dBm
Upper Bound: -74.94 dBm

Total Measurements: 16829
Potential Outliers: 113
Outlier Percentage: 0.67 %
```



In [30]:

```
# Step 27: Prepare Features and Target for ML

# Select input features
features = [
    "Latitude",
    "Longitude",
    "Data Throughput (Mbps)",
    "Latency (ms)",
    "Network Type"
]

# Select target variable
target = "Signal Strength (dBm)"

# Create feature matrix and target vector
X = df[features].copy()
y = df[target].copy()

# Display dataset information
print("Feature matrix shape:", X.shape)
print("Target vector shape:", y.shape)

print("\nSelected features:")
print(X.dtypes)

print("\nTarget variable:", target)

print("\nTarget statistics:")
print(y.describe())
```

```
Feature matrix shape: (16829, 5)
Target vector shape: (16829,)

Selected features:
Latitude                  float64
Longitude                 float64
Data Throughput (Mbps)    float64
Latency (ms)              float64
Network Type               object
dtype: object

Target variable: Signal Strength (dBm)

Target statistics:
count    16829.000000
mean       -90.072484
std          5.399368
min       -116.942267
```

```
25%        -93.615962
50%        -89.665566
75%        -86.145491
max        -74.644848
Name: Signal Strength (dBm), dtype: float64
```

In [31]:

```
import sklearn
print("Scikit-learn version:", sklearn.__version__)
```

```
Scikit-learn version: 1.9.1
```

In [32]:

```
# Step 28: Feature Encoding and Standardization

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

# Separate numerical and categorical features
numeric_features = [
    "Latitude",
    "Longitude",
    "Data Throughput (Mbps)",
    "Latency (ms)"
]

categorical_features = ["Network Type"]

# Numerical preprocessing
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Categorical preprocessing
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

# Combine both preprocessing pipelines
preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, numeric_features),
    ("categorical", categorical_pipeline, categorical_features)
])

print("Preprocessing pipeline created successfully.")
print("Numerical features:", numeric_features)
print("Categorical features:", categorical_features)
```

```
Preprocessing pipeline created successfully.
Numerical features: ['Latitude', 'Longitude', 'Data Throughput (Mbps)',
'Latency (ms)']
Categorical features: ['Network Type']
```

In [33]:

```python
# Step 29: Train Random Forest Regression Model

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Create the complete ML pipeline
model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", RandomForestRegressor(
        n_estimators=100,
        random_state=42,
        n_jobs=-1
    ))
])

# Train the model
model.fit(X_train, y_train)

print("Random Forest model trained successfully!")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Number of input features:", X.shape[1])
```

```
Random Forest model trained successfully!
Training samples: 13463
Testing samples: 3366
Number of input features: 5
```

In [34]:

```python
# Step 30: Evaluate Random Forest Model

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
import numpy as np

# Predict signal strength for test data
y_pred = model.predict(X_test)

# Calculate evaluation metrics
mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(
    mean_squared_error(y_test, y_pred)
)

r2 = r2_score(y_test, y_pred)

# Display results
print("Random Forest Regression Evaluation")
print("-----------------------------------")
print("MAE:", round(mae, 3), "dB")
print("RMSE:", round(rmse, 3), "dB")
print("R² Score:", round(r2, 4))
```

```
Random Forest Regression Evaluation
-----------------------------------
MAE: 3.249 dB
RMSE: 4.134 dB
R² Score: 0.3953
```

In [35]:

```
# Step 31: Actual vs Predicted Signal Strength

plt.figure(figsize=(10, 6))

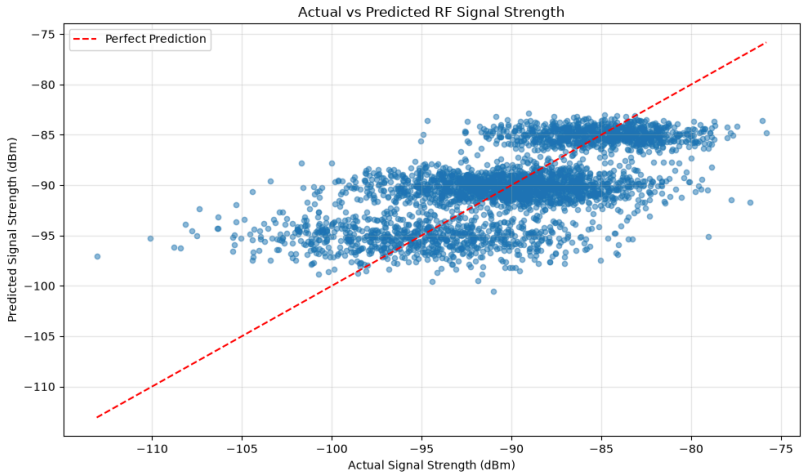
plt.scatter(
    y_test,
    y_pred,
    alpha=0.5,
    s=20
)

# Reference line for perfect predictions
lower = min(y_test.min(), y_pred.min())
upper = max(y_test.max(), y_pred.max())

plt.plot(
    [lower, upper],
    [lower, upper],
    linestyle="--",
    color="red",
    label="Perfect Prediction"
)

plt.title("Actual vs Predicted RF Signal Strength")
plt.xlabel("Actual Signal Strength (dBm)")
plt.ylabel("Predicted Signal Strength (dBm)")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()

plt.show()
```



In [36]:

```
# Step 32: Residual Analysis

# Calculate residuals
residuals = y_test - y_pred

# Plot residual distribution
plt.figure(figsize=(10, 6))

sns.histplot(
    residuals,
    bins=30,
    kde=True
)

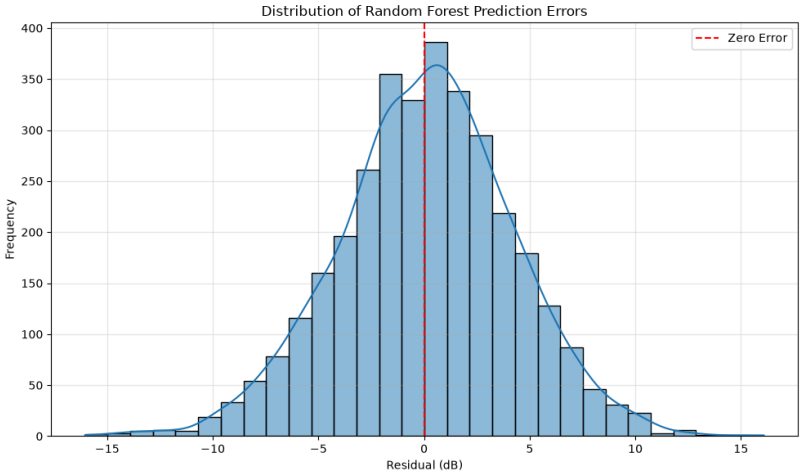
plt.axvline(
    0,
    color="red",
    linestyle="--",
    label="Zero Error"
)

plt.title("Distribution of Random Forest Prediction Errors")
plt.xlabel("Residual (dB)")
plt.ylabel("Frequency")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
```

```
plt.show()

# Display residual statistics
print("Residual Analysis")
print("-----------------")
print("Mean residual:", round(residuals.mean(), 3), "dB")
print("Median residual:", round(residuals.median(), 3), "dB")
print("Residual standard deviation:",
      round(residuals.std(), 3), "dB")

print("Positive residuals:", (residuals > 0).sum())
print("Negative residuals:", (residuals < 0).sum())
print("Zero residuals:", (residuals == 0).sum())
```




```
Residual Analysis
-----------------
Mean residual: 0.093 dB
Median residual: 0.221 dB
Residual standard deviation: 4.134 dB
Positive residuals: 1755
Negative residuals: 1611
Zero residuals: 0
```

In [37]:

```
# Step 33: Baseline Model Comparison

from sklearn.dummy import DummyRegressor
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
import numpy as np
import pandas as pd

# Create a simple baseline model
baseline_model = DummyRegressor(strategy="mean")

# Train baseline using training data only
baseline_model.fit(X_train, y_train)

# Generate predictions on the same test set
baseline_pred = baseline_model.predict(X_test)

# Evaluate baseline
baseline_mae = mean_absolute_error(y_test, baseline_pred)

baseline_rmse = np.sqrt(
    mean_squared_error(y_test, baseline_pred)
)

baseline_r2 = r2_score(y_test, baseline_pred)

# Compare both models
comparison = pd.DataFrame({
    "Metric": ["MAE (dB)", "RMSE (dB)", "R² Score"],
    "Random Forest": [mae, rmse, r2],
    "Baseline Model": [
        baseline_mae,
        baseline_rmse,
        baseline_r2
    ]
})

print("Model Performance Comparison")
print("----------------------------")
print(comparison.round(4).to_string(index=False))
```

```
Model Performance Comparison
----------------------------
   Metric  Random Forest  Baseline Model
 MAE (dB)         3.2495          4.2540
RMSE (dB)         4.1340          5.3163
 R² Score         0.3953         -0.0001
```

In [38]:

```
# Step 34: Spatial Block Validation
```

```
from sklearn.model_selection import GroupShuffleSplit
from sklearn.base import clone
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
import pandas as pd

# Align coordinates with the selected feature rows
coords = df.loc[X.index, ["Latitude", "Longitude"]]

# Create larger spatial blocks (0.1-degree grid)
lat_block = np.floor(coords["Latitude"] / 0.1).astype(int)
lon_block = np.floor(coords["Longitude"] / 0.1).astype(int)

spatial_groups = (
    lat_block.astype(str) + "_" + lon_block.astype(str)
)

# Split entire geographical blocks between train and test
splitter = GroupShuffleSplit(
    n_splits=1,
    test_size=0.2,
    random_state=42
)

train_idx, test_idx = next(
    splitter.split(X, y, groups=spatial_groups)
)

X_train_block = X.iloc[train_idx]
X_test_block = X.iloc[test_idx]
y_train_block = y.iloc[train_idx]
y_test_block = y.iloc[test_idx]

# Train a fresh pipeline
block_model = clone(model)
block_model.fit(X_train_block, y_train_block)

# Predict on held-out blocks
y_pred_block = block_model.predict(X_test_block)

# Evaluate
block_mae = mean_absolute_error(y_test_block, y_pred_block)
block_rmse = np.sqrt(
    mean_squared_error(y_test_block, y_pred_block)
)
block_r2 = r2_score(y_test_block, y_pred_block)

print("SPATIAL BLOCK VALIDATION")
print("------------------------")
print("Block size: 0.1 degrees")
print("Training samples:", len(train_idx))
print("Testing samples:", len(test_idx))
print("Training blocks:",
      spatial_groups.iloc[train_idx].nunique())
print("Testing blocks:",
      spatial_groups.iloc[test_idx].nunique())

print("\nMAE:", round(block_mae, 4), "dB")
print("RMSE:", round(block_rmse, 4), "dB")
print("R² Score:", round(block_r2, 4))
```

```
SPATIAL BLOCK VALIDATION
------------------------
Block size: 0.1 degrees
Training samples: 14044
Testing samples: 2785
Training blocks: 14
Testing blocks: 4

MAE: 3.2696 dB
RMSE: 4.1753 dB
R² Score: 0.3679
```

In [39]:

```
# Step 35: Feature Importance Analysis

from sklearn.inspection import permutation_importance
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate feature importance on the original test set
importance = permutation_importance(
    model,
    X_test,
    y_test,
    n_repeats=5,
    random_state=42,
    scoring="r2",
    n_jobs=-1
)

# Create feature importance dataframe
feature_importance = pd.DataFrame({
    "Feature": X_test.columns,
    "Importance": importance.importances_mean
})
```

```
# Sort by importance
feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

# Display results
print("RF Feature Importance")
print("---------------------")
print(feature_importance.to_string(index=False))

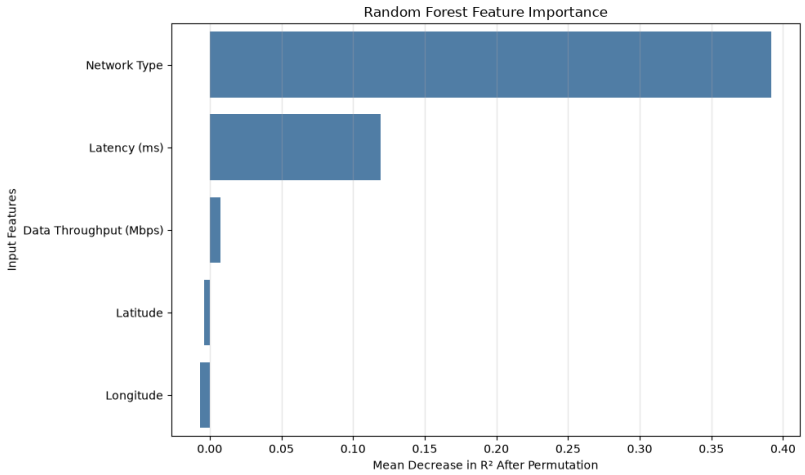
# Visualize feature importance
plt.figure(figsize=(10, 6))

sns.barplot(
    data=feature_importance,
    x="Importance",
    y="Feature",
    color="steelblue"
)

plt.title("Random Forest Feature Importance")
plt.xlabel("Mean Decrease in R² After Permutation")
plt.ylabel("Input Features")
plt.grid(axis="x", alpha=0.3)
plt.tight_layout()

plt.show()
```

```
RF Feature Importance
---------------------
               Feature  Importance
          Network Type    0.391771
          Latency (ms)    0.119284
Data Throughput (Mbps)    0.007642
              Latitude   -0.003977
             Longitude   -0.006867
```



In [40]:

```
# Step 36: Compare Model With and Without Network Type

from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
import numpy as np
import pandas as pd

# 1. Train model using all five features
full_model = clone(model)
full_model.fit(X_train, y_train)
full_pred = full_model.predict(X_test)

# 2. Remove Network Type
X_train_num = X_train.drop(columns=["Network Type"])
X_test_num = X_test.drop(columns=["Network Type"])

# 3. Create numerical-only preprocessing pipeline
numeric_features = X_train_num.columns.tolist()
```

```
numeric_preprocessor = ColumnTransformer([
    ("numeric", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]), numeric_features)
])

# 4. Train model using numerical features only
numeric_model = Pipeline([
    ("preprocessor", numeric_preprocessor),
    ("regressor", RandomForestRegressor(
        n_estimators=100,
        random_state=42,
        n_jobs=-1
    ))
])

numeric_model.fit(X_train_num, y_train)
numeric_pred = numeric_model.predict(X_test_num)

# 5. Calculate evaluation metrics
def evaluate_model(actual, predicted):
    return {
        "MAE (dB)": mean_absolute_error(actual, predicted),
        "RMSE (dB)": np.sqrt(
            mean_squared_error(actual, predicted)
        ),
        "R² Score": r2_score(actual, predicted)
    }

all_features = evaluate_model(y_test, full_pred)
without_network = evaluate_model(y_test, numeric_pred)

# 6. Compare results
comparison = pd.DataFrame({
    "Metric": list(all_features.keys()),
    "All Features": list(all_features.values()),
    "Without Network Type": list(without_network.values())
})

print("STEP 36: FEATURE COMPARISON")
print("----------------------------")
print(comparison.round(4).to_string(index=False))
```

```
STEP 36: FEATURE COMPARISON
----------------------------
   Metric  All Features  Without Network Type
 MAE (dB)        3.2495                3.5914
RMSE (dB)        4.1340                4.5123
 R² Score        0.3953                0.2796
```

In [41]:

```
# Step 37: Final ML Results Summary (Dynamic)

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.dummy import DummyRegressor
import numpy as np

print("=" * 60)
print("FINAL MACHINE LEARNING RESULTS")
print("=" * 60)

# 1. Dataset details
print("\n1. DATASET")
print(f"Total samples: {len(df)}")
print(f"Input features: {X.shape[1]}")
print(f"Target variable: {target}")

# 2. Random Forest performance
final_pred = model.predict(X_test)

rf_mae = mean_absolute_error(y_test, final_pred)
rf_rmse = np.sqrt(mean_squared_error(y_test, final_pred))
rf_r2 = r2_score(y_test, final_pred)

print("\n2. RANDOM FOREST PERFORMANCE")
print(f"MAE: {rf_mae:.4f} dB")
print(f"RMSE: {rf_rmse:.4f} dB")
print(f"R² Score: {rf_r2:.4f}")

# 3. Baseline comparison
baseline = DummyRegressor(strategy="mean")
baseline.fit(X_train, y_train)
baseline_pred = baseline.predict(X_test)

base_mae = mean_absolute_error(y_test, baseline_pred)
base_rmse = np.sqrt(mean_squared_error(y_test, baseline_pred))
base_r2 = r2_score(y_test, baseline_pred)

mae_reduction = (
    (base_mae - rf_mae) / base_mae * 100
    if base_mae != 0 else np.nan
)
rmse_reduction = (
    (base_rmse - rf_rmse) / base_rmse * 100
    if base_rmse != 0 else np.nan
)
```

```
print("\n3. BASELINE COMPARISON")
print(f"Baseline MAE: {base_mae:.4f} dB")
print(f"Baseline RMSE: {base_rmse:.4f} dB")
print(f"Baseline R²: {base_r2:.4f}")
print(f"MAE reduction: {mae_reduction:.2f}%")
print(f"RMSE reduction: {rmse_reduction:.2f}%")

print("\n4. MODEL FINDINGS")
print("Use the permutation-importance results from Step 35 to identify the most
influential features.")
print(f"Random Forest R²: {rf_r2:.4f}")

print("\n5. VALIDATION")
print("Random train-test evaluation: completed")
print("Spatial validation: refer to the results from Step 34")

print("\n" + "=" * 60)
```

```
============================================================
FINAL MACHINE LEARNING RESULTS
============================================================

1. DATASET
Total samples: 16829
Input features: 5
Target variable: Signal Strength (dBm)

2. RANDOM FOREST PERFORMANCE
MAE: 3.2495 dB
RMSE: 4.1340 dB
R² Score: 0.3953

3. BASELINE COMPARISON
Baseline MAE: 4.2540 dB
Baseline RMSE: 5.3163 dB
Baseline R²: -0.0001
MAE reduction: 23.61%
RMSE reduction: 22.24%

4. MODEL FINDINGS
Use the permutation-importance results from Step 35 to identify the most
influential features.
Random Forest R²: 0.3953

5. VALIDATION
Random train-test evaluation: completed
Spatial validation: refer to the results from Step 34

============================================================
```

In [42]:

```
# Step 38: Export the Trained Model for Streamlit

from pathlib import Path
import joblib

# Identify the project root
project_root = Path.cwd()

if project_root.name == "notebooks":
    project_root = project_root.parent

# Create the models folder
model_dir = project_root / "models"
model_dir.mkdir(parents=True, exist_ok=True)

# Save the complete trained pipeline
model_path = model_dir / "rf_signal_model.joblib"
joblib.dump(model, model_path)

print("Model exported successfully!")
print("Saved location:", model_path.resolve())
```

```
Model exported successfully!
Saved location: C:\4-1\DSDV\RF_Project\models\rf_signal_model.joblib
```

In [44]:

```python
# Step 39: Export Held-Out Test Records

from pathlib import Path
import pandas as pd

project_root = Path.cwd()

if project_root.name == "notebooks":
    project_root = project_root.parent

model_dir = project_root / "models"
model_dir.mkdir(parents=True, exist_ok=True)

test_records = X_test.copy()
test_records["Actual Signal Strength (dBm)"] = y_test

test_records.to_csv(
    model_dir / "rf_test_records.csv",
    index=False
)

print("Test records exported:", len(test_records))
print("Saved to:", model_dir / "rf_test_records.csv")
```

```
Test records exported: 3366
Saved to: c:\4-1\DSDV\RF_Project\models\rf_test_records.csv
```

In [ ]:

```python
# Step 40: Compressing the model to reduce file size for deployment

from pathlib import Path
import joblib
import os

# Find the model whether your notebook is in the root or notebooks folder
project_root = Path.cwd()
if project_root.name == "notebooks":
    project_root = project_root.parent

model_path = project_root / "models" / "rf_signal_model.joblib"
compressed_path = project_root / "models" / "rf_signal_model_compressed.joblib"

if not model_path.exists():
    raise FileNotFoundError(f"Model not found: {model_path}")

# Load the saved model and write a compressed copy
model = joblib.load(model_path)
joblib.dump(model, compressed_path, compress=3)

original_mb = os.path.getsize(model_path) / (1024 * 1024)
compressed_mb = os.path.getsize(compressed_path) / (1024 * 1024)

print(f"Original model:   {original_mb:.2f} MB")
print(f"Compressed model: {compressed_mb:.2f} MB")
print(f"Space saved:      {original_mb - compressed_mb:.2f} MB")
print(f"Saved to:         {compressed_path}")
```

```
Original model:   116.99 MB
Compressed model: 31.91 MB
Space saved:      85.07 MB
Saved to:         c:\4-
1\DSDV\RF_Project\models\rf_signal_model_compressed.joblib
```

In [ ]:

```python
# Step 41: Load and Test the Compressed Model

import joblib

test_model = joblib.load(compressed_path)

print("Compressed model loaded successfully!")
print("Model type:", type(test_model).__name__)
```

```
Compressed model loaded successfully!
Model type: Pipeline
```

In [ ]:

```python
# Step 42: Compare Predictions of Original and Compressed Models

import numpy as np

original_predictions = model.predict(X_test)
compressed_predictions = test_model.predict(X_test)

print(
    "Predictions unchanged:",
    np.allclose(original_predictions, compressed_predictions)
)

print(
    "Maximum prediction difference:",
    np.max(np.abs(original_predictions - compressed_predictions))
)
```

```
Predictions unchanged: True
Maximum prediction difference: 5.684341886080802e-14
```